Escuela Profesional de Ciencias de la Computación

Curso: Estructuras de Datos Avanzadas

PRÁCTICA DE LABORATORIO 5

Analizador Sintáctico Predictivo LL(1) SQL

Tema:	Especificación de Trabajo Práctico	**Puntaje:**	20 puntos
Lenguaje:	C++	**Modalidad:**	Grupal

1. Introducción y Contexto General

El objetivo de este proyecto práctico es la construcción progresiva de un servidor de base de datos relacional para sentencias de tipo `SELECT`.

El presente trabajo corresponde a la segunda parte de la Fase 2 y se centra en el diseño e implementación del **analizador sintáctico predictivo non-recursive LL(1)**. Este componente recibe la secuencia ordenada de tokens generada por el analizador léxico (Práctica 4) y se encarga de validar si dicha secuencia cumple estrictamente con las reglas de la gramática definida por el equipo.

2. Objetivo del Trabajo

Diseñar formalmente una Gramática Libre de Contexto (CFG) apta para el análisis LL(1), calcular sus conjuntos *FIRST* y *FOLLOW*, construir la Tabla de Parseo LL(1) e implementar en C++ el motor de reconocimiento sintáctico basado en pila para un subconjunto de sentencias SQL, detectando y reportando cualquier error sintáctico con precisión.

3. Planteamiento Técnico

Cada equipo de trabajo deberá definir y formalizar el alcance de su propio subconjunto del lenguaje SQL. A partir de este alcance, el equipo diseñará su gramática formal y garantizará que cumpla las condiciones requeridas para ser procesada mediante un analizador predictivo LL(1).

El analizador sintáctico consumirá los tokens directamente desde la interfaz de salida del analizador léxico desarrollado en la Práctica 4.

3.1. Diseño Libre de la Gramática LL(1) por Equipo

Cada grupo propondrá su propia especificación gramatical en Notación BNF o EBNF. La gramática propuesta debe cumplir obligatoriamente las siguientes restricciones técnicas:

1. **Ausencia de recursión a la izquierda:** Debe eliminarse cualquier tipo de recursión izquierda (directa o indirecta) para evitar bucles infinitos en el análisis predictivo.
2. **Factorización a la izquierda:** Se deben factorizar los prefijos comunes entre producciones de un mismo no terminal para evitar ambigüedades en la elección de la regla.
3. **Condición LL(1) estricta:** Para cualquier símbolo no terminal A con producciones alternativas $A \to \alpha \mid \beta$, los conjuntos de selección (FIRST($\alpha \cdot$ FOLLOW(A))) deben ser mutuamente disjuntos. No deben existir conflictos en la Tabla de Parseo.

3.1.1. Requisitos Mínimos del Subconjunto SQL a Soportar

Aunque el diseño de las producciones es libre, la gramática del grupo deberá ser capaz de derivar como mínimo las siguientes estructuras:

- Cláusula `SELECT` con proyección de todas las columnas (`*`) o listas de columnas explicitadas y separadas por comas.
- Cláusula `FROM` indicando la tabla origen.
- Cláusula `WHERE` opcional que admita expresiones de comparación relacional ($=, <>, <, >, <=, >=$) y operadores lógicos (`AND`, `OR`).
- Delimitador explícito de fin de sentencia (por ejemplo, `;`).

4. Componentes Principales

La solución en C++ deberá estructurarse en los siguientes componentes modulares:

- **Cálculo / Especificación de Conjuntos LL(1):** Documentación matemática de los conjuntos FIRST(X) y FOLLOW(X) para cada no terminal de la gramática propuesta.
- **Tabla de Parseo LL(1):** Matriz $M[A, a]$ que mapea cada par (No Terminal, Terminal) a una regla de producción específica.
- **Motor de Análisis Basado en Pila:** Algoritmo iterativo LL(1) que gestiona la pila de símbolos sintácticos, procesa el token actual (*lookahead*) y realiza las operaciones de emparejamiento (*match*) o expansión de producciones.
- **Manejo de Errores Sintácticos:** Detección de tokens no esperados o celdas vacías en la tabla de parseo, indicando la línea, columna, token encontrado y los tokens que se esperaban en ese estado.
- **Trazador / Registro del Parseo:** Mecanismo de salida por consola que muestre el estado paso a paso de la pila sintáctica, la entrada restante y la regla de producción aplicada en cada paso.

4.1. Requisitos Funcionales

El motor de parseo deberá cumplir con los siguientes puntos:

1. Integrarse sin modificaciones destructivas con el módulo Léxico de la Práctica 4.
2. Utilizar una pila sintáctica explícita (análisis predictivo no recursivo).
3. Representar formalmente la Tabla de Parseo LL(1) dentro del código C++ (mediante tablas de dispersión, arreglos bidimensionales o estructuras de mapeo).
4. Determinar de forma unívoca la aceptación o rechazo sintáctico de la sentencia de entrada.
5. Rechazar sentencias sintácticamente inválidas sin colapsar el programa (*crash* o *panic*).

5. Especificación de Entregables

Para completar satisfactoriamente el trabajo, los equipos deberán presentar los siguientes entregables:

Entregable	Componentes clave	Criterio de aceptación
1. Formalización de la Gramática	EBNF de la gramática propuesta, cálculo explicativo de FIRST y FOLLOW.	Gramática libre de recursión izquierda, factorizada y demostrada como LL(1) sin conflictos.
2. Tabla de Parseo	Representación matricial $M[A, a]$ documentada e implementada.	Mapeo unívoco para cada combinación no terminal y token de entrada.
3. Motor LL(1) por Pila	Código en C++ del parser predictivo no recursivo e integración léxica.	Procesa la cadena de tokens manipulando la pila sintáctica correctamente hasta el estado de aceptación.
4. Manejo de Errores	Reporte de fallos sintácticos con posición y contexto de tokens.	Detecta entradas inválidas e informa la línea, columna y tokens esperados.
5. Batería de Pruebas	Pruebas unitarias para las reglas de la gramática del equipo.	Muestra la ejecución y la traza de la pila para consultas válidas e inválidas.
6. Informe PDF	Documento formal con la gramática, tabla LL(1), traza de la pila y resultados.	Documento subido al classroom siguiendo la estructura y normas del curso.

6. Pruebas y Validación

Cada equipo deberá diseñar un conjunto de pruebas adaptado a la gramática que ha definido. Como mínimo, las pruebas deberán incluir:

1. **Consultas válidas simples:** Verificación de sentencias con la estructura mínima definida por el grupo.
2. **Consultas válidas complejas:** Proyecciones múltiples, combinaciones de cláusulas opcionales (`WHERE`) y expresiones compuestas.
3. **Simulación paso a paso de la Pila:** Documentación en el informe de la traza completa de la pila sintáctica (paso, pila, entrada, acción) para al menos una consulta válida.
4. **Detección de errores sintácticos:** Pruebas con palabras clave fuera de orden, omisión de tokens obligatorios (como el nombre de la tabla o el delimitador final) y expresiones mal formadas.

7. Entrega en GitHub e Informe

Toda la solución deberá estar integrada en el repositorio de GitHub de la Fase 2 del grupo, dentro de una carpeta específica para la Práctica 5 (`/parser_ll1`). El **Informe PDF** con el diseño completo de la gramática y el análisis LL(1) deberá adjuntarse en el repositorio e incluirse en el envío correspondiente.

8. Rúbrica de Calificación

La evaluación se realizará sobre una escala de 0 a 20 puntos, distribuida según los siguientes criterios:

Criterio	**Descripción del desempeño**	**Puntaje**
Revisión de Primer Avance	Presentación y revisión intermedia durante el laboratorio	2.0 pts
Diseño y Análisis Gramatical LL(1)	Eliminación de recursión a la izquierda, factorización y cálculo correcto final de FIRST y FOLLOW.	3.0 pts
Tabla de Parseo y Algoritmo LL(1)	Correcta representación de la tabla $M[A, a]$, implementación del algoritmo de pila en C++ e integración con el Lexer.	7.0 pts
Tratamiento de Errores y Pruebas	Calidad en el reporte de errores sintácticos, cobertura de pruebas para sentencias válidas/inválidas y claridad en la traza de la pila.	4.0 pts
Informe PDF y Repositorio	Documentación formal clara, inclusión del desarrollo paso a paso, orden del código en GitHub y cumplimiento de formatos.	4.0 pts
Calificación Total		**20.0 pts**